강화학습 교육 3일차 Report
Q-Learning의 한계와 Deep Q-Network

이름:김도훈
학번:2026406041
학과:로봇학부

<목차>
1. 코드 분석
2. 결과 분석

1. 코드 분석

MDP 정의

상태: 위치, 속도, 각도, 각속도
행동: 엡실론 그리디 정책
상태 전이: CartPole 내부 상태 전이 규칙
보상: 매 step 마다 + 1, 최대 500
할인율: 0.95

Q-Network 구현(online, target)

```
def make_net():
    return nn.Sequential(
        nn.Linear(4, 128),
        nn.ReLU(),
        nn.Linear(128, 128),
        nn.ReLU(),
        nn.Linear(128, 2)
    )
```

DQN은 기존 Q-Leaning과 다르게 Q-table을 사용하지 않고 인공신경망을 이용하여 Q-value를 예측한다. 입력이 4개(위치, 속도, 각도, 각속도)이고 출력 2개인 MLP를 구현하였다.
DQN은 Q-value를 예측하고 가중치를 학습하는 online network와 TD-loss 계산 시 사용되는 목표값을 계산하는 target network가 따로 존재한다. 그래서 코드에서도

```
online = make_net()
target = make_net()
target.load_state_dict(online.state_dict())
target.eval()
```

이와 같이 구현하였다. 맨 위에서 생성한 신경망의 구조를 이용해 두개의 신경망 생성했고 초기 가중치는 동일하게 설정했다.

Replay Buffer

```
MEMORY_SIZE = 20000 # Replay Buffer 사이즈
memory = deque(maxlen=MEMORY_SIZE)
```

위 코드는 Replay buffer를 생성한 코드이다. MEMORY SIZE를 20000로 설정해 최대 20000개의 경험을 저장할 수 있도록 했다. 하나의 경험은 다음과 같이 구성된다.

memory.append((state.copy(), action, reward, ns.copy(), term))

(상태, 행동, 보상, 다음 상태, 종료 여부)로 구성 되어있다. 이 Replay Buffer는 연속된 경험의 상관 관계를 줄이고 과거에 수집한 경험을 반복하기 위해서 사용되었다.

batch = random.sample(memory, BATCH_SIZE)

엡실론 그리디 정책

```
def select_action(state, epsilon):
    if random.random() < epsilon:
        return env.action_space.sample()
    with torch.no_grad():
        x = torch.as_tensor(
            state,
            dtype=torch.float32
        ).unsqueeze(0)
        action = online(x).argmax(dim=1).item()
    return action
```

위 코드는 엡실론 그리디 정책을 구현한 코드이다. 이 엡실론 그리디 정책은 학습 시 신경망이 예측한 Q-value만을 이용해서 행동을 선택하면 특정 행동에 치우칠 수 있기 때문에 이 정책을 사용해 탐험과 활용 사이의 균형을 맞춘 것이다. 또한 학습 시에는 엡실론을 0.05까지 감소시킨다.

epsilon = max(0.05, 1 - 0.95 * (step - 1) / 15000)

학습 루프

```
    # Online Network 예측 Q-value
    prediction = online(S).gather(1, A).squeeze(1)
    # Target Network로 TD Target 계산
    with torch.no_grad():
        next_q = target(NS).max(dim=1).values
```

```
        y = R + GAMMA * (1 - T) * next_q
    # TD Loss
    loss = loss_fn(prediction, y)
```

학습 루프에서는 위에서 생성한 online net과 taget net을 사용해 실제 Q-Value를 예측하고 목표값을 계산하다. 또한 TD-Loss를 계산하시 위해서는 MSE 손실 함수와 MAE 손실 함수를 결합한 SmoothL1Loss 손실 함수를 사용한다. 이 함수를 사용함으로써 작은 오차에는 제곱 오차 형태를 사용하고 큰 오차에는 선형적인 형태를 사용하므로 큰 오차에 대한 민감도를 완화할 수 있디.

```
if step % TARGET_UPDATE == 0:
    target.load_state_dict( online.state_dict() )
```

위 코드를 사용해 target net의 가중치를 online net의 가중치로 500 step 마다 갱신 하도록 했다.

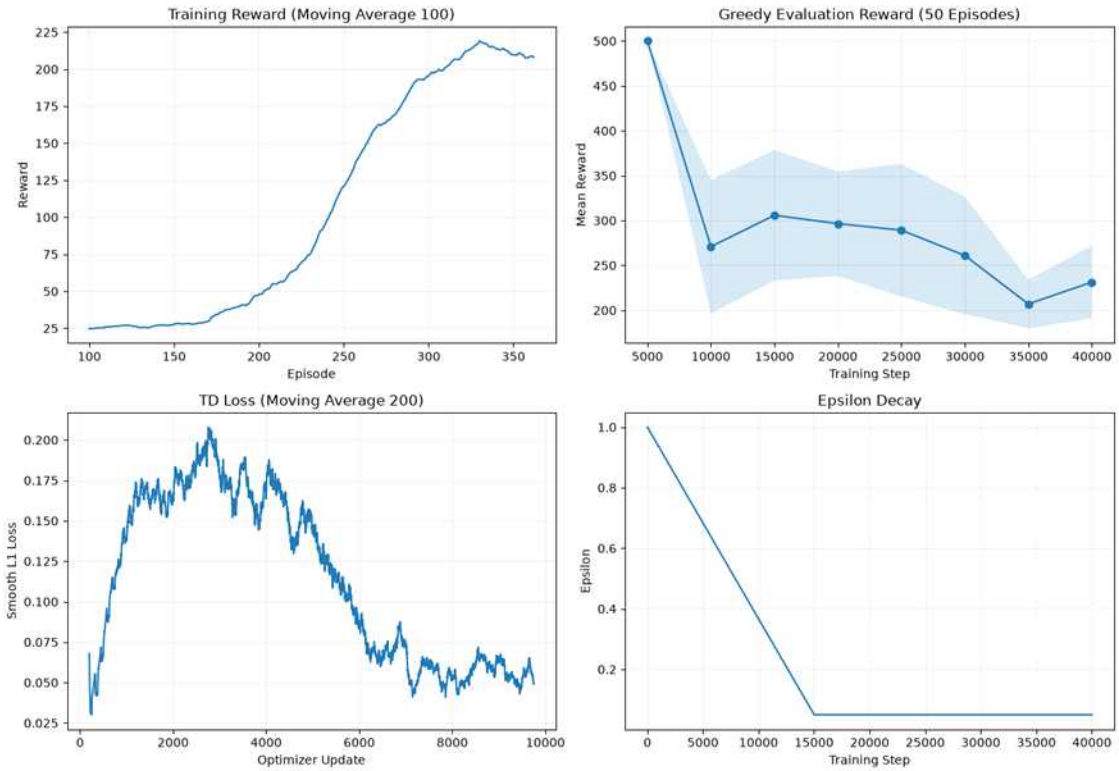
2. 결과 분석

왼쪽 위 그래프: DQN으로 cartpole 환경에서 학습하면서 각 Episode에서 받은 누적 보상 그래프
왼쪽 아래 그래프: TD Loss와 online Network의 가중치 수정 간의 관계를 나타내는 그래프
오른쪽 위 그래프: 학습된 DQN 모델로 5000 step 마다 50회 평가했을 때 받은 누적 보상 그래프
오른쪽 아래 그래프: 학습을 하면서 앱실론의 변화를 보여주는 그래프

먼저 왼쪽 위 그래프부터 분석을 해보면 초기에는 평균 누적 보상이 25이었지만 학습이 진행되면서 최대 220까지는 증가한 것을 확인할 수 있다. 이것은 DQN이 CartPole 환경에서 막대의 균형을 잡기 위한 방향으로 학습하고 있는 것으로 분석한다.

두 번째 왼쪽 아래 그래프를 분석하면 초반에 Optimizer Update (online network의 가중치 수정 횟수)가 증가할수록 TD-Loss가 감소하는 것이 아니라 초반에 폭증하고 후반부로 갈수록 감소하는 경향을 나타내고 있다.

세 번째 오른쪽 위 그래프를 분석해보면 학습된 DQN 모델로 5000 step 마다 50회 평가를 진행했을 때 점점 누적 보상이 감소하고 있다. 이것은 학습이 진행 될수록 DQN 모델의 성능이 점점 감소하고 있다는 것이다.



네 번째 그래프는 엡실론이 40000 step 학습을 진행할 동안 1에서 15000 step이 되었을 때 0.05로 감소하고 그 이후로는 유지되는 것을 확인할 수 있다.

이 네 가지 그래프를 분석해보았을 때 몇 가지 궁금증이 생겼다. 왜 TD-Loss가 갑자기 증폭하다가 감소할까? 그리고 왜 TD-Loss가 감소함에 따라 Greedy Evaluation Reward 그래프는 왜 감소할까?

먼저 이 두 가지 궁금증을 해결하기 위해서는 TD-Loss와 Greedy Evaluation Reward 그래프가 감소하는 것이 어떤 것을 의미하는지 알 필요가 있다. DQN에서 TD-Loss는 target network에서 계산한 Q-Value의 목표값과 online network에서 계산한 Q-Value의 예측값의 차이이다. Greedy Evaluation Reward 그래프가 감소하는 것은 학습 횟수가 증가할수록 CartPole 제어 성능이 감소한다는 것이다.

첫 번째 궁금증에 대한 나의 분석은 학습 초기에는 target net과 online net 둘 다 충분히 학습되지 않은 상태이고 학습을 시작하면 target net의 가중치는 online net의 가중치로 step 500이 되었을 때 갱신된다. 그래서 그래프에서 TD-Loss가 초반에 증폭했던 이유는 학습 초기에는 학습이 충분히 되지 않은 상태의 가중치로 target net에서 목표값을 계산했고 step 500이 되었을 때는 어느 정도 학습이 된 가중치로 갱신되고 목표값을 계산했기 때문에 예측값과 차이가 큰 목표값으로 계산되었을 것이다. 그러므로 TD-Loss가 초기에 증폭했던 것으로 분석한다. 이 현상은 DQN이 잘 학습하고 있다는 증거로 볼 수 있다.

두 번째 궁금증에 대한 나의 분석은 먼저 TD-Loss와 평가 보상 간의 관계는 없다. 그래서 Greedy Evaluation Reward 그래프는 왜 감소하는지만 분석하면 첫 번째 궁금증에 대한 분석과 비슷한 원인으로 분석한다. DQN은 신경망의 가중치를 TD_Loss

가 감소하는 방향으로 학습한다. TD-Loss는 target net에서 계산한 Q-Value 목표값과 online net에서 계산한 Q-Value의 예측값의 차이로 구한다. target net은 계산한 목표값은 정확한 정답이 아니라 추정값이므로 오차가 존재한다. 이 오차가 학습 횟수 5000~10000 사이에 크게 발생했을 것이고 이 오차에 의해 TD-Loss가 잘못 계산되어 online net의 가중치에도 악영향을 줬을 것이다. 그래서 정책의 성능이 감소했던 것으로 분석한다.

===== DQN Greedy =====	===== Random Policy =====
Total Episodes: 50	Total Episodes: 50
Mean Reward: 231.30	Mean Reward: 20.82
Std Reward: 40.47	Std Reward: 10.56
Min Reward: 181.00	Min Reward: 8.00
Max Reward: 396.00	Max Reward: 51.00

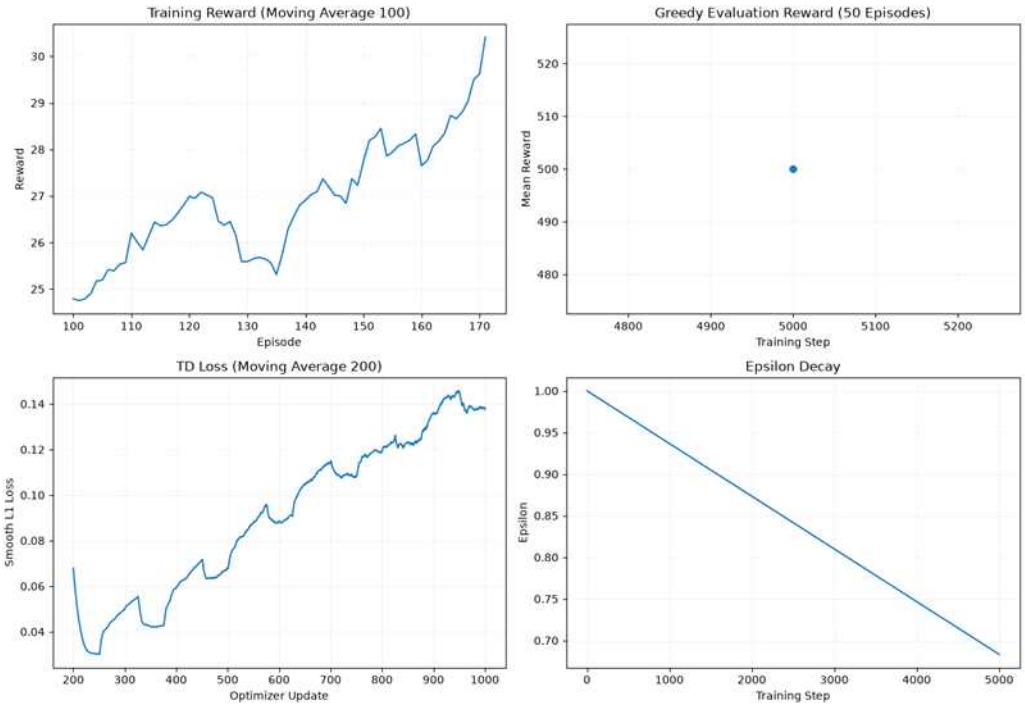
또한 무작위 DQN 학습된 Greedy 정책과 무작위 정책을 비교해 본 결과 당연히 CartPole 제어 성능은 DQN으로 학습된 Greedy 정책이 훨씬 뛰어났다. 이유는 Greedy 정책은 Q-Value 값이 큰 쪽으로 행동하기 때문이다.

3. 비교 분석

40000 step 동안 학습한 DQN은 CartPole 제어 성능이 좋지 않았다. 그래서 5000 step으로 변경 후 DQN을 다시 학습해보았다. 그 결과

===== DQN Greedy =====
Total Episodes: 50
Mean Reward: 500.00
Std Reward: 0.00
Min Reward: 500.00
Max Reward: 500.00

50회 평가 모두 최대 보상 500을 받았다. 실제로 CartPole과 같은 간단한 게임은 500~1000회 정도의 학습 횟수를 권장한다고 한다.


Training Reward (Moving Average 100)
Reward
Episode
Greedy Evaluation Reward (50 Episodes)
Mean Reward
Training Step
TD Loss (Moving Average 200)
Smooth L1 Loss
Optimizer Update
Epsilon Decay
Epsilon
Training Step